



Bangladesh University of Engineering and Technology
Department of Electrical and Electronic Engineering
Dhaka-1205, Bangladesh

Communication Systems I Laboratory

EEE 310

PROJECT REPORT

RFID-Based Smart Voting System Using Arduino

Prepared By:

Mushfiqur Rahman ID: 2106181	Chayan Paul ID: 2106182	Kayes Sami Malitha ID: 2106184
---------------------------------	----------------------------	-----------------------------------

Submitted to:

Dr. Md. Forkan Uddin
Professor, Department of EEE, BUET

Sadat Tahmeed Azad
Lecturer, Department of EEE, BUET

Objective:

The goal of this project is to develop a secure, efficient, and user-friendly electronic voting system utilizing **RFID(Radio-Frequency Identification)** technology. This system is intended to serve as a digital replacement for traditional paper-based voting methods, with the following key features:

- ☐ Improves interaction between the voter and the system by providing clear, real-time feedback.
- ☐ Authenticates voters through RFID card verification.
- ☐ Ensures that each individual can vote only once.
- ☐ Streamlines the processes of vote casting and result counting.
- ☐ Enhances transparency while minimizing human errors.

Introduction about RFID Technology in Context to Our Project:

Radio Frequency Identification (RFID) is a wireless communication technology used for automatic identification and data exchange between a reader and a tag. It enables secure and contactless interaction, making it highly suitable for applications that require fast, reliable authentication—such as an electronic voting system. To achieve this, the project incorporates the RC522 RFID module, a compact and cost-effective reader that operates at 13.56 MHz and communicates with a microcontroller (e.g., Arduino) via the SPI protocol. It is widely used in embedded systems and is ideal for short-range authentication tasks like voter verification.

Components Used:

List of all components and their explanations that have been used in our project are given below-

1. RFID Reader

The RC522 RFID reader module is a widely used, low-cost RFID reader that operates at 13.56 MHz, designed for short-range communication with passive RFID tags and cards based on the ISO/IEC 14443A standard. It is ideal for applications like access control, attendance systems, and electronic voting projects. The module is known for its compact size, fast response, and easy integration with microcontrollers like Arduino, ESP32, or Raspberry Pi.



Fig: RFID-RC522

The RC522 communicates using the SPI protocol, which ensures fast and reliable data exchange. It includes a built-in copper coil antenna that creates the electromagnetic field necessary to power and read passive RFID tags within a typical range of 2–5 cm.

The functions of the different pins of the RFID reader are given below:

- ☐ **SDA (SS)** – Slave Select: Used to enable communication with the microcontroller over SPI.
- ☐ **SCK** – Serial Clock: Provides the clock signal for SPI communication.
- ☐ **MOSI** – Master Out Slave In: Sends data from the microcontroller to the RFID module.
- ☐ **MISO** – Master In Slave Out: Sends data from the RFID module back to the microcontroller.
- ☐ **IRQ** – Interrupt Request: Can be used to alert the microcontroller when a tag is detected (optional).
- ☐ **GND** – Ground: Connects to the ground of the power source.

- ❑ **3.3V** – Power Supply: Powers the module (important: do not connect to 5V, as it can damage the module).
- ❑ **RST** – Reset: Used to reset the module during operation.

When a valid RFID card or tag is brought near the module, the RC522 reads its unique ID (UID) and sends it to the microcontroller, which can then use that data for identification, access control, or logging.

2. RFID Tag

An RFID tag is a small device used to store and transmit data wirelessly when activated by an RFID reader. It consists of two main parts: a microchip, which holds a unique identification number (UID) and possibly other data, and an antenna, which communicates with the reader. In most basic systems, like those using the RC522 module, passive RFID tags are used. These tags have no battery and are powered by the electromagnetic field emitted by the reader. When brought close to the reader (typically within 2–5 cm), the tag transmits its UID, allowing the system to identify the item or person carrying it.



Fig: RFID Tag

3. Arduino Uno

The Arduino Uno is a popular, beginner-friendly microcontroller board based on the ATmega328P chip. It is widely used in electronics projects for sensing, control, and automation. The Uno can be programmed using the Arduino IDE via a USB connection.



Fig: Arduino Uno

The functions of the different pins of the Arduino Uno are given below:

- **Digital Pins (0–13):** Used for digital input/output (e.g., turning an LED on/off). Pins **0 (RX)** and **1 (TX)** are also used for serial communication.
- **PWM Pins (~3, 5, 6, 9, 10, 11):** Can output analog-like signals using Pulse Width Modulation (useful for dimming LEDs or controlling motors).
- **Analog Pins (A0–A5):** Used to read analog sensors (e.g., temperature, light). They return values from 0 to 1023.
- **Power Pins:**
 - **5V:** Powers external components.
 - **3.3V:** For 3.3V devices (like RC522 RFID module).
 - **GND:** Ground.
 - **VIN:** Voltage input when powering from an external source.
- **AREF:** Reference voltage for analog inputs.
- **Reset:** Resets the microcontroller.

4. LCD (16x2) with I2C Module

A 16x2 LCD is a display module that can show 2 lines of 16 characters each. When used with an I2C module (usually based on the PCF8574 chip), it becomes much easier to connect to a microcontroller like Arduino, using only two wires (SDA and SCL) instead of 6–10 pins.

The I2C module is a small board soldered onto the back of the LCD, allowing communication through the I2C protocol.



Fig: LCD with I2C

Pin functions of the I2C module:

- ☐ **VCC** – Power supply (usually **5V** from Arduino).
- ☐ **GND** – Ground connection.
- ☐ **SDA (Serial Data)** – Data line for I2C communication (connects to **A4** on Arduino Uno).
- ☐ **SCL (Serial Clock)** – Clock line for I2C (connects to **A5** on Arduino Uno).

Working:

- ☐ The I2C module sends commands to the LCD using only two Arduino pins.
- ☐ Using the **LiquidCrystal_I2C** library in Arduino to control the display (print text, move cursor, etc.).
- ☐ It also includes a potentiometer to adjust display contrast and may have a jumper to control the backlight.

This setup is perfect for saving I/O pins and simplifying wiring in projects like temperature displays, clocks, voting systems, etc.

5. Push Buttons

A 4-pin push button is a simple switch used to control circuits manually. It has two pairs of internally connected pins-pressing the button connects the two pairs, allowing current to flow. Commonly used in breadboards or with microcontrollers like Arduino for user input.



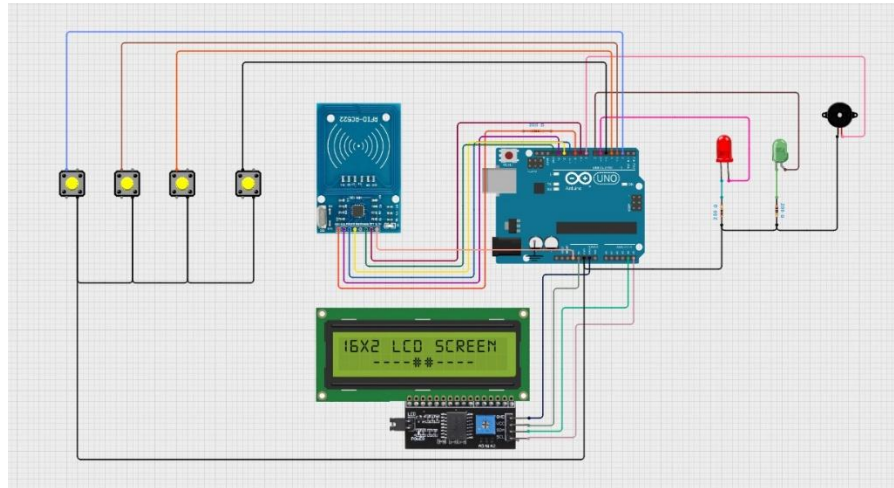
Fig: Push Button

Besides this components we have also used some-

- ☐ Jumper wires
- ☐ 2 Leds
- ☐ 2 Resistors
- ☐ 1 Buzzer and
- ☐ Breadboard

Circuit Diagram:

Here is the circuit diagram for our project-



The voter scans their RFID tag using the RFID reader to initiate the voting process. Based on the provided circuit diagram and the programmed logic, the system then carries out the voting operation, which will be explained in the following sections.

Working Principal:

- ❑ Firstly, there comes a message in the LCD that “Show Your ID Card”.



- ❑ Then the person holding the RFID tag scans their card using the RFID reader. If the UID matches a valid entry in the system, confirming the person as an authentic voter, they are then allowed to cast their vote.



- ❑ If the person is not an authentic voter the system will say “Unauthorized Access” and he can't vote.



- ❑ In our project, there are three candidates named A, B, and C. The voter can choose to vote for any one of them using the 3 push buttons. After the vote is cast, the system displays the selected candidate.



- ❑ If a person who has already voted tries to vote again, the system will deny access and display the message: 'You have already voted.'



- ❑ If a person enters the voting section but does not cast a vote within 10 seconds, the system will display a 'Timeout' message, and the voting attempt will be cancelled.



- ❑ After the voting period ends, the authority can terminate the voting process using a special Authority Card, which then allows the system to display the final results.



- ❑ A special push button (4th one) is provided to display the winner of the election. When pressed, the system reveals the winning candidate—clearly here, candidate A is the winner.



- ❑ If two or more candidates receive the same highest number of votes, the system will declare a tie between them, and no winner will be announced.



- ❑ Once the voting process has ended, the system automatically resets, preparing itself for the next voting event.

This is how our smart voting machine using RFID technology functions.

Code:

```
#include <SPI.h>
```

```
#include <MFRC522.h>
```

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
MFRC522 mfrc522(10, 9); // SS_PIN=10, RST_PIN=9
```

```
// Pin definitions
```

```
#define LED_Y 7
```

```
#define LED_R 6
```

```
#define BUZZER 8
```

```
#define v1 2
```

```
#define v2 3
```

```
#define v3 4
```

```

#define v4 5

// Voting and state variables

int vote1 = 0, vote2 = 0, vote3 = 0;

bool votedFlags[6] = { false }; // Flags for 6 voters

// Known UIDs

const String VOTER_UIDS[6] = {
    "92 2B FA 03", "D3 11 6A AD", "93 0D F9 03",
    "D2 B6 34 03", "E3 3F F9 03", "79 92 FA 03"
};

const String AUTHORITY_UID = "53 4C 9A 36";

void setup() {
    Serial.begin(9600);
    SPI.begin();
    mfrc522.PCD_Init();
    lcd.init();
    lcd.backlight();
    pinMode(LED_Y, OUTPUT);
    pinMode(LED_R, OUTPUT);
    pinMode(BUZZER, OUTPUT);
    noTone(BUZZER);

    // Enable internal pullups for buttons
    pinMode(v1, INPUT_PULLUP);
    pinMode(v2, INPUT_PULLUP);
    pinMode(v3, INPUT_PULLUP);
    pinMode(v4, INPUT_PULLUP);
}

```

```

void loop() {
    // Wait for new card
    if (!mfrc522.PICC_IsNewCardPresent()) {
        displayIdleScreen();
        return;
    }

    // Read card
    if (!mfrc522.PICC_ReadCardSerial()) return;

    // Get UID
    String content = "";
    for (byte i = 0; i < mfrc522.uid.size; i++) {
        content.concat(String(mfrc522.uid.uidByte[i] < 0x10 ? " 0" : " "));
        content.concat(String(mfrc522.uid.uidByte[i], HEX));
    }
    content.toUpperCase();

    // Halt PICC to prevent repeated reads
    mfrc522.PICC_HaltA();
    mfrc522.PCD_StopCrypto1();

    lcd.clear();

    // Authority card handling
    if (content.substring(1) == AUTHORITY_UID) {
        handleAuthorityCard();
        delay(1000);
        return;
    }
}

```

```

    }
// Voter card handling
for (int i = 0; i < 6; i++) {
    if (content.substring(1) == VOTER_UIDS[i]) {
        if (!votedFlags[i]) {
            handleVoter(i);
        } else {
            displayAlreadyVoted();
        }
        return;
    }
}

// Unauthorized card
displayUnauthorized();
}

// Helper functions
void displayIdleScreen() {
    lcd.setCursor(3, 0);
    lcd.print("SHOW YOUR");
    lcd.setCursor(4, 1);
    lcd.print("ID CARD");
}

void handleVoter(int voterIndex) {
    lcd.setCursor(4, 0);
    lcd.print("Welcome");
    tone(BUZZER, 500, 500);
    delay(2000);
}

```

```
lcd.clear();

// Wait for vote with 10s timeout
unsigned long startTime = millis();
while (millis() - startTime < 10000) {
    lcd.setCursor(4, 0);
    lcd.print("Vote Now");
    lcd.setCursor(0, 1);
    lcd.print("1->A 2->B 3->C");
    if (digitalRead(v1) == LOW) {
        votedFlags[voterIndex] = true;
        recordVote(1, "A");
        return;
    }
    if (digitalRead(v2) == LOW) {
        votedFlags[voterIndex] = true;
        recordVote(2, "B");
        return;
    }
    if (digitalRead(v3) == LOW) {
        votedFlags[voterIndex] = true;
        recordVote(3, "C");
        return;
    }
}

lcd.clear();
lcd.print("Vote Timeout");
```

```
digitalWrite(LED_R, HIGH);  
tone(BUZZER, 300);  
delay(1000);  
noTone(BUZZER);  
digitalWrite(LED_R, LOW);  
delay(1000);  
lcd.clear();  
}
```

```
void recordVote(int candidate, String label) {
```

```
    lcd.clear();  
    digitalWrite(LED_Y, HIGH);  
    if (candidate == 1) vote1++;  
    if (candidate == 2) vote2++;  
    if (candidate == 3) vote3++;
```

```
    lcd.print("Voted for");  
    lcd.setCursor(0, 1);  
    lcd.print("Candidate " + label);  
    tone(BUZZER, 700, 200);  
    delay(2000);  
    lcd.clear();  
    digitalWrite(LED_Y, LOW);
```

```
// Debounce release
```

```
while (digitalRead(v1) == LOW || digitalRead(v2) == LOW || digitalRead(v3) == LOW)  
    ;
```

```
    delay(100);  
}  
  
void displayAlreadyVoted() {  
    lcd.print("You already");  
    lcd.setCursor(0, 1);  
    lcd.print("voted");  
    digitalWrite(LED_R, HIGH);  
    tone(BUZZER, 300, 1000);  
    delay(2000);  
    digitalWrite(LED_R, LOW);  
    lcd.clear();  
}
```

```
  
void handleAuthorityCard() {  
    lcd.print("Authority");  
    digitalWrite(LED_Y, HIGH);  
    tone(BUZZER, 500, 300);  
    delay(2000);  
    lcd.clear();  
  

```

```
// Display votes
```

```
lcd.setCursor(1, 0);  
lcd.print("A");  
lcd.setCursor(1, 1);  
lcd.print(vote1);  
lcd.setCursor(5, 0);  
lcd.print("B");
```

```

lcd.setCursor(5, 1);
lcd.print(vote2);
lcd.setCursor(9, 0);
lcd.print("C");
lcd.setCursor(9, 1);
lcd.print(vote3);

// Wait for result/continue button
while ((digitalRead(v4) == HIGH) && (digitalRead(v1) == HIGH));

if (digitalRead(v4) == LOW) {
    determineWinner();
    resetSystem();
} else if (digitalRead(v1) == LOW) {
    lcd.clear();
    lcd.print("Vote Continue");
    delay(2000);
    lcd.clear();
}

digitalWrite(LED_Y, LOW);
}

void determineWinner() {
    int totalVotes = vote1 + vote2 + vote3;
    lcd.clear();
    if (totalVotes == 0) {
        lcd.print("No Votes Polled");
    } else if (vote1 > vote2 && vote1 > vote3) {

```

```

    lcd.setCursor(5, 0);
    lcd.print("A Wins");
} else if (vote2 > vote1 && vote2 > vote3) {
    lcd.setCursor(5, 0);
    lcd.print("B Wins");
} else if (vote3 > vote1 && vote3 > vote2) {
    lcd.setCursor(5, 0);
    lcd.print("C Wins");
} else if (vote1 == vote2 && vote1 > vote3) {
    lcd.print("A-B Tie");
} else if (vote1 == vote3 && vote1 > vote2) {
    lcd.print("A-C Tie");
} else if (vote2 == vote3 && vote2 > vote1) {
    lcd.print("B-C Tie");
} else {
    lcd.print("Tie");
}
delay(2000);
lcd.clear();
}

void resetSystem() {
    vote1 = vote2 = vote3 = 0;
    delay(1000);
    lcd.print("System Reset");
    delay(2000);
    lcd.clear();
    for (int i = 0; i < 6; i++) {

```

```
    votedFlags[i] = false;
}
}

void displayUnauthorized() {
    lcd.print("UNAUTHORIZED");
    lcd.setCursor(0, 1);
    lcd.print("ACCESS");
    digitalWrite(LED_R, HIGH);
    tone(BUZZER, 300, 1000);
    delay(1000);
    digitalWrite(LED_R, LOW);
    lcd.clear();
}
```

Code Explanation

This section concisely explains the Arduino code for the RFID-based voting system. The code integrates various hardware components, including the RFID module, LCD display, push buttons, LEDs and buzzer, to create a secure and user-friendly voting process. It ensures only authorized users can vote, prevents multiple voting attempts, and offers real-time feedback, all essential for an efficient electronic voting experience.

1. Setting Up Libraries and Hardware

The code starts by including a few important libraries that allow the Arduino to communicate with different components. Specifically:

- ❑ SPI.h and MFRC522.h are used to handle communication with the RFID card reader.
- ❑ Wire.h is needed for I2C communication.
- ❑ LiquidCrystal_I2C.h helps control the LCD display.

The LCD is initialized with the I2C address 0x27 and is configured to have 2 rows and 16 columns, which makes it suitable for showing short and clear messages to the user during voting.

Additionally, the RFID reader is connected to the Arduino using pin 10 as SS (Slave Select) and pin 9 as RST (Reset). This setup ensures the system is ready to detect RFID cards before any voting begins. This initial configuration is essential, as it gets all the hardware components ready to work together smoothly in the voting process.

2. Pin Definitions and Variables

The code assigns specific pins to components like LEDs, a buzzer, and four push buttons used for voting. It also defines variables like `vote1`, `vote2`, and `vote3` to store the number of votes for each candidate. An array `votedFlags[6]` tracks whether each of the six registered voters has already voted. This arrangement ensures that the system can track votes accurately while preventing double voting.

3. UID Management

A critical security feature is the use of unique identifiers (UIDs) for each RFID card. The system stores six UIDs for authorized voters and one UID for the authority figure. When a card is scanned, the UID is compared with the stored values to determine whether access should be granted. This step is vital for ensuring that only registered individuals can participate in the voting process.

4. Setup Function

The `setup()` function initializes all components. It starts the serial monitor, sets up the RFID reader and LCD, and configures the button and LED pins. Internal pull-up resistors are activated for the buttons to ensure reliable input detection. This function gets the entire system ready to detect users and start voting.

5. Main Loop

The core logic of the system lies in the `loop()` function, which continuously scans for RFID cards. If a card is detected, its UID is read and matched against the stored list. If it matches the authority card, the system displays vote counts and provides the option to declare results. If it matches a voter who hasn't voted, they are allowed to cast a vote. If they have already voted, a message notifies them. Unrecognized cards are denied access. This flow guarantees that each voter can only vote once.

6. Voting Procedure

When an eligible voter is identified, the `handleVoter()` function is called. It welcomes the voter via the LCD and gives them 10 seconds to press one of the voting buttons. Buttons `v1`, `v2`, and `v3` correspond to candidates A, B, and C respectively. Once a button is pressed, the vote is recorded, and the voter is marked as having voted, blocking any future attempts.

7. Vote Recording

The `recordVote()` function is responsible for increasing the vote count for the selected candidate. It also activates a buzzer and lights an LED to give the user confirmation that their vote was counted. To avoid accidental multiple votes, the system waits until the button is released before proceeding. This adds a layer of reliability to the voting process.

8. Authority Functions

When the authority scans their special RFID card, the system shows the current vote tally on the LCD. The authority can press `v4` to declare the winner or `v1` to return to the voting session. This ensures that only the designated person can access sensitive result information, maintaining election integrity.

9. Winner Declaration

This function compares the vote counts of candidates A, B, and C. It determines which candidate has the highest number of votes and displays the result on the LCD. If there is a tie, that information is also shown. This makes the system transparent in its result declaration.

10. System Reset

The `resetSystem()` function clears all vote counts and voter flags, effectively resetting the system for a new election round. This is useful for reusing the system without manual reprogramming, making it sustainable for repeated use.

11. Security and Feedback Features

Security is a top priority in this system. Unauthorized RFID cards are blocked from voting. Each registered voter can vote only once, thanks to the `votedFlags[]` mechanism. The LCD provides real-time updates and instructions, while the buzzer and LEDs confirm actions, enhancing usability and trust. This code ensures a secure, anonymous, and reliable electronic voting experience. Only eligible voters can participate, and no one can vote more than once. Even the authority cannot see whom a specific person voted for, preserving complete voter privacy. Only the final tally and winner are revealed, ensuring fairness in the democratic process.

Advantages:

- ❑ **Secure and Tamper-Proof:** The system ensures that votes cannot be changed or tampered with, making the election fair and trustworthy.
- ❑ **Prevents Double Voting:** Each voter can vote only once because the system remembers who has already voted, stopping any repeated or fraudulent votes.
- ❑ **Easy to Use:** Voters just need to scan their RFID card and press a button to vote. The clear LCD messages guide them through the process, making it simple for everyone.
- ❑ **Portable and Convenient:** The whole setup is compact and can be moved easily, so it can be used in different places without hassle.
- ❑ **Real-Time Vote Counting:** Votes are counted instantly as they are cast, allowing the authority to check the current results anytime without delay.
- ❑ **Only Authorized Voters Can Participate:** Unauthorized users cannot vote, which helps keep the election secure and reliable.
- ❑ **Voter Privacy Protected:** No one, not even the election authority, can see who a voter chose, ensuring that every vote remains secret.
- ❑ **Clear Result Declaration:** The winner or tie results are displayed clearly on the screen, making the outcome transparent to everyone involved.

Application of Communication Systems in the RFID-Based Voting Machine:

- ❑ **RFID Communication:** The system uses RFID technology, which relies on radio frequency communication to transfer data wirelessly between the RFID tag and the reader. This is a core example of short-range wireless communication.
- ❑ **Data Exchange Protocols:** The Arduino communicates with the RFID reader module via communication protocols such as SPI (Serial Peripheral Interface) or UART (Universal Asynchronous Receiver-Transmitter), which are fundamental serial communication methods in embedded systems.

- ❑ **Information Processing and Transmission:** Once the RFID tag data (UID) is read, the Arduino processes and transmits this data internally to other components like displays or storage modules, involving communication between devices.
- ❑ **Authentication & Security:** The communication link between the tag and reader ensures secure data exchange, verifying voter identity, which is a critical aspect of communication systems.

This project demonstrates practical use of wireless and serial communication principles, connecting RFID wireless data transfer and embedded device communication to enable a smart, secure voting system.

Future Improvements:

- ❑ **Data Storage on EEPROM or SD Card**
To ensure votes are not lost during a power failure, voting data can be saved on permanent memory like EEPROM or an SD card.
- ❑ **Online Result Display (IoT Integration)**
Connecting the system to the internet can allow real-time vote results to be displayed on a website, making it easier to monitor elections remotely.
- ❑ **Integration with National ID (NID) System**
By linking RFID with NID cards in Bangladesh, only verified citizens can vote, preventing fake or duplicate voters.
- ❑ **Biometric Verification (Fingerprint or Face Recognition)**
Adding fingerprint or face ID verification will stop people from using someone else's card to vote, ensuring more secure and fair elections.
- ❑ **Support for Multiple Elections and Regions**
The system can be upgraded to handle more than one election at a time or cover various areas, making it suitable for large-scale or national use.

Conclusion:

The RFID-based smart voting machine developed in this project offers a secure, efficient, and user-friendly approach to electronic voting. By using RFID cards for voter identification, the system ensures that only registered voters can cast a single vote, thereby eliminating the risk of duplicate or unauthorized voting. The integration of an LCD display provides clear instructions and real-time feedback, making the system easy to operate for users with minimal technical knowledge.

Security is further enhanced by restricting access to voting results and system reset functionality to an authorized RFID card holder, ensuring the integrity and confidentiality of the voting process. The use of visual (LEDs) and audio (buzzer) signals strengthens communication between the system and its users, contributing to a smoother and more transparent experience.

In summary, this project demonstrates how microcontroller-based systems, combined with RFID technology, can create a reliable and tamper-resistant voting platform suitable for educational institutions, small communities, and controlled environments.